Module 1: Container Security Fundamentals — Assessment Questions

---

Module-Level Assessment (3 MCQs)

Question 1: Namespace Isolation

A security analyst discovers that a container process running as PID 1 inside its namespace is visible as PID 4521 on the host. The analyst also observes that the container's `/proc/1/ns/pid` inode differs from the host's `/proc/1/ns/pid` inode. What does this indicate?

A) The container is running in privileged mode and has bypassed PID isolation
B) The container is in a separate PID namespace; its PID 1 maps to a different PID on the host, confirming process ID isolation is active
C) The container has a kernel-level hypervisor providing hardware-based isolation
D) The container's PID namespace is misconfigured and leaking process information to the host

Correct Answer: B

Explanation: Linux PID namespaces provide process ID isolation — a process that appears as PID 1 inside its namespace is mapped to a different PID (e.g., 4521) on the host. Different inode numbers for the `/proc/[pid]/ns/pid` symlinks confirm that the processes are in separate PID namespaces. This is normal, expected behavior — not a misconfiguration. The host can always see containerized processes (containers are just processes), but the container cannot see host processes. Privileged mode (A) does not change PID namespace behavior. Containers do not use hypervisors (C).

Question 2: Docker Daemon Security

During a security audit, you discover the following in `/etc/docker/daemon.json`:

```
{
  "hosts": ["unix:///var/run/docker.sock", "tcp://0.0.0.0:2375"],
  "icc": true
}
```

Which of the following represents the MOST critical security risk in this configuration?

A) Inter-container communication (ICC) being enabled, allowing containers to send traffic to each other
B) The Docker daemon listening on TCP port 2375 on all interfaces without TLS, which provides unauthenticated root-equivalent access to anyone who can reach the port
C) Using a Unix socket for local communication, which could be exploited by local users
D) The absence of resource limits in the daemon configuration, exposing the host to denial-of-service attacks

Correct Answer: B

Explanation: Exposing the Docker daemon on `tcp://0.0.0.0:2375` without TLS authentication is the most critical risk. Port 2375 is the unauthenticated Docker API port. Anyone who can reach it over the network — including from the internet if the host is publicly accessible — can execute arbitrary Docker commands, including running privileged containers with the host filesystem mounted. This is functionally equivalent to unauthenticated root access. While ICC enabled (A) is a concern for lateral movement, and Unix socket access (C) requires local access, the TCP exposure is remote and unauthenticated, making it categorically more severe. Resource limits (D) are a best practice but are not configured in daemon.json for individual containers.

Question 3: Shared Kernel Risk

Your organization runs 15 containers across 3 hosts. A critical Linux kernel vulnerability (CVE) is announced that allows privilege escalation from an unprivileged process to root. Which statement BEST describes the impact and required response?

A) Only containers running as root are affected; containers running as non-root users are safe and no patching is needed for them
B) All 15 containers across all 3 hosts are potentially vulnerable because they share the host kernel; all 3 hosts must be patched and rebooted regardless of container configuration
C) Rebuilding and redeploying container images with updated base images is sufficient to remediate the vulnerability
D) Applying a Seccomp profile to all containers will fully mitigate the kernel vulnerability without requiring a host kernel patch

Correct Answer: B

Explanation: Containers share the host kernel. A kernel vulnerability affects every process running on that kernel — including all containers, regardless of their configuration, user, or capabilities. Patching requires updating the host kernel and rebooting the host (or using live patching where supported). Non-root containers (A) may be harder to exploit but are not inherently safe from a kernel privilege escalation vulnerability. Rebuilding container images (C) only changes the userspace; it does not change the kernel. Seccomp profiles (D) can reduce the attack surface by blocking certain system calls, but cannot patch a vulnerability in a system call that must remain available.

Final Assessment Contribution (2 Comprehensive MCQs)

Final Question 1: Defense-in-Depth Scenario

You are designing a container security architecture for a financial services company that processes credit card data (PCI-DSS scope). The application uses microservices deployed in Docker containers on shared hosts. Which combination of controls provides the MOST comprehensive security posture?

A) Use privileged containers with AppArmor profiles, enable Docker Content Trust, and scan images weekly
B) Run containers with `--cap-drop=ALL`, enable user namespaces, deploy on VMs with one trust zone per VM, enforce Seccomp profiles, scan images in CI/CD pipeline, and use mutual TLS between services
C) Use Kubernetes with default pod security settings, enforce network policies, and run all containers

as non-root
D) Deploy gVisor as the container runtime on bare metal, disable Seccomp (since gVisor provides its own kernel), and rely on image scanning for vulnerability management

Correct Answer: B

Explanation: Option B implements defense-in-depth across multiple layers: dropping all capabilities reduces the syscall attack surface, user namespaces map container root to an unprivileged host user, VM-level isolation between trust zones satisfies PCI-DSS segmentation requirements, Seccomp profiles restrict system calls, CI/CD image scanning catches vulnerabilities before deployment, and mutual TLS secures service-to-service communication. Option A contradicts itself — privileged containers bypass AppArmor's protections. Option C relies on Kubernetes defaults which are insufficiently restrictive for PCI-DSS. Option D disables Seccomp, removing a valuable defense layer, and bare metal without VM isolation is unlikely to satisfy PCI-DSS segmentation requirements.

Final Question 2: Attack Surface Analysis

An attacker has gained command execution inside a container through a web application vulnerability. The container is running with default Docker settings (no additional hardening). Which of the following attack paths is MOST likely to succeed for the attacker to escalate to host-level access?

A) Exploiting a vulnerability in the container's PID namespace to directly access host processes, since PID namespaces have known bypass techniques
B) Using the 14 default Linux capabilities and access to the host kernel's 300+ system calls to exploit a kernel vulnerability, since default containers lack comprehensive syscall filtering
C) Connecting to the Docker daemon through the container's network interface, since Docker API is accessible from within containers by default
D) Loading a malicious kernel module from within the container, since kernel modules are accessible by default to all containers

Correct Answer: B

Explanation: With default Docker settings, containers have 14 Linux capabilities and access to 300+ system calls (Docker's default Seccomp profile blocks only ~44 calls). This provides a substantial attack surface against the shared host kernel. If a kernel vulnerability exists in any of the permitted system calls, the attacker can exploit it for privilege escalation and container escape. PID namespace bypasses (A) are not a common attack vector — the namespace is kernel-enforced. The Docker daemon (C) is not accessible from within containers by default — it listens on a Unix socket on the host, not on the container's network. Loading kernel modules (D) requires the SYS_MODULE capability, which is not in the default capability set — this is only possible in privileged containers.